
Part B

Ex: 8**Title:** Optimal Road Routes

Problem Description: You are the chief planner for a transportation department in a country with several cities. The cities are connected by a network of roads, each with a specific travel time (in hours). Your task is to determine the shortest travel time from a designated source city to all other cities.

Method: Represent the problem in the form of a graph using adjacency matrix or adjacency list. Using the Single Source Shortest Path Algorithm find the shortest path from the source city to all other cities.

Theory Reference: Module 3**Algorithm**

```

ALGORITHM Dijkstra(G, s)
    //Dijkstra's algorithm for single-source shortest paths
    //Input: A weighted connected graph  $G = (V, E)$  with nonnegative weights
    //      and its vertex  $s$ 
    //Output: The length  $d_v$  of a shortest path from  $s$  to  $v$ 
    //      for every vertex  $v$  in  $V$ 
    Initialize( $Q$ ) //initialize priority queue to empty
    for every vertex  $v$  in  $V$ 
         $d_v \leftarrow \infty$ ;  $p_v \leftarrow \text{null}$ 
    Insert( $Q, v, d_v$ ) //initialize vertex priority in the priority queue
     $d_s \leftarrow 0$ ; Decrease( $Q, s, d_s$ ) //update priority of  $s$  with  $d_s$ 
     $V_T \leftarrow \emptyset$ 
    for  $i \leftarrow 0$  to  $|V| - 1$  do
         $u^* \leftarrow \text{DeleteMin}(Q)$  //delete the minimum priority element
         $V_T \leftarrow V_T \cup \{u^*\}$ 
        for every vertex  $u$  in  $V - V_T$  that is adjacent to  $u^*$  do
            if  $d_{u^*} + w(u^*, u) < d_u$ 
                 $d_u \leftarrow d_{u^*} + w(u^*, u)$ ;  $p_u \leftarrow u^*$ 
                Decrease( $Q, u, d_u$ )
  
```

Program

```

import java.util.Scanner;

public class Dijkstras
{
    static int min(int m, int n)
  
```

```
{  
    return(( m < n) ? m: n);  
}  
  
static void dijkstra(int cost[][],int source,int n)  
{  
    int[] tvertex=new int[n];  
    int dist[]=new int[n];  
    int min,u=0,v,i,j;  
    System.arraycopy(cost[source], 0, dist, 0, n);  
    //Initialize dist from source to source as 0  
    //mark source vertex - estimated for its shortest path  
    tvertex[source] = 1;  
    dist[source] = 0;  
    for(i=0; i < n-1; i++)  
    {  
        //Find the nearest neighbour vertex  
        min = Integer.MAX_VALUE;  
        for( j = 0; j < n; j++)  
        {  
            if (tvertex[j] == 0 && dist[j]<min)  
            {  
                min = dist[j];  
            }  
        }  
    }  
}
```

```
        u = j;
    }
}

tvertex[u]=1;

//Update the shortest path of neighbour of u
for(v=0;v<n;v++)
{
    if(tvertex[v]==0 && cost[u][v]!=Integer.MAX_VALUE)
    {
        dist[v]= min(dist[v],dist[u]+cost[u][v]);
    }
}
}

System.out.println(" The shortest travel time ");

for(v=0; v<n; v++)

    System.out.println("from "+source+" city to "+v+" city is " + dist[v]);
}

public static void main(String[] args)
{
    int source,n;

    Scanner s=new Scanner(System.in);
```

```
System.out.println("Enter the no.of cities");

n = s.nextInt();

int cost[][] = new int[n][n];

System.out.println("Enter the travel time in hours between every two cities");

for(int i=0; i<n; i++)
{
    for (int j=0; j<n; j++)
    {
        cost[i][j] = s.nextInt();

        if (cost[i][j] == 0)

            cost[i][j] = Integer.MAX_VALUE;

    }
}

System.out.println("Enter the source city");

source = s.nextInt();

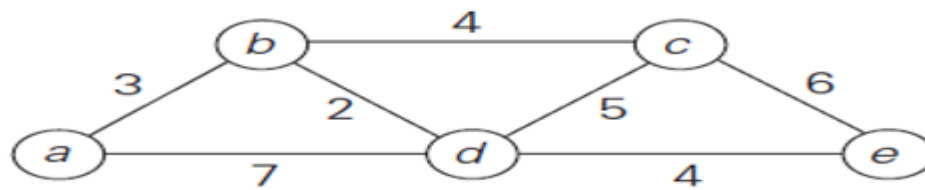
dijkstra(cost,source,n);

}

}
```

Output

1.



0 3 0 7 0

3 0 4 2 0

0 4 0 5 6

7 2 5 0 4

0 0 6 4 0

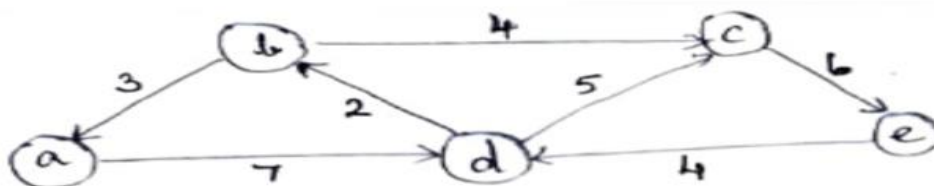
```

run:
Enter the no.of cities
5
Enter the travel time in hours between every two cities
0 3 0 7 0
3 0 4 2 0
0 4 0 5 6
7 2 5 0 4
0 0 6 4 0

Enter the source city
0
The shortest travel time
from 0 city to 0 city is 0
from 0 city to 1 city is 3
from 0 city to 2 city is 7
from 0 city to 3 city is 5
from 0 city to 4 city is 9
BUILD SUCCESSFUL (total time: 20 seconds)

```

2.



0 0 0 7 0

3 0 4 0 0

0 0 0 0 6

0 2 5 0 0

0 0 0 4 0

Consider source as a.

Cost matrix

0 0 0 7 0 999 999 999 7 999

3 0 4 0 0 3 999 4 999 999

0 0 0 0 6 999 999 999 999 6

0 2 5 0 0 999 2 5 999 999

0 0 0 4 0 999 999 999 4 999

```
run:
Enter the no.of cities
5
Enter the travel time in hours between every two cities
0 0 0 7 0
3 0 4 0 0
0 0 0 0 6
0 2 5 0 0
0 0 0 4 0

Enter the source city
0
The shortest travel time
from 0 city to 0 city is 0
from 0 city to 1 city is 9
from 0 city to 2 city is 12
from 0 city to 3 city is 7
from 0 city to 4 city is 18
BUILD SUCCESSFUL (total time: 23 seconds)
```